

HUNBN, a 16-nm Digital In-Memory-Compute SoC for Edge CNN Application Achieving 24 TOPs/W (4b) at System Level

Weijie Jiang¹, Graduate Student Member, IEEE, Cédric Caron², Prabhat Avasare, Member, IEEE, Marc Pauwels, Marian Verhelst¹, Fellow, IEEE, and Wim Dehaene¹, Senior Member, IEEE

Abstract—This work presents HUNBN, a fully digital in-memory-compute (DIMC) system-on-chip (SoC) implemented using 16-nm FinFET CMOS technology. Designed for edge convolution neural network (CNN) applications, the DIMC-based architecture enhances energy efficiency by significantly reducing the energy associated with static random access memory (SRAM). At the macro level, three key techniques are employed to optimize energy and area efficiency: 1) the adoption of a foundry-pushed rule 6T bit-cell array; 2) a split MAC workflow; and 3) synchronous DIMC operation. At the system level, the data movement is tailored to efficiently support both CNN and transposed CNN layers. The proposed design achieves 126 TOPs/W (4-bit MAC) at the macro level and 24 TOPs/W at the system level, while offering a memory density of 508 kB/mm² and a compute density of 1.27 K MAC units/mm². This enables a highly energy-efficient, scalable platform for edge CNN applications.

Index Terms—Convolution neural network (CNN), digital, in-memory-compute (IMC), static random access memory (SRAM), system-on-chip (SoC), U-shape network (U-Net).

I. INTRODUCTION

IN-MEMORY-COMPUTE (IMC) is gaining increasing attention from both academia and industry because of its potential for higher energy efficiency, while alleviating the memory wall bottleneck [1], [2], [3], [4], [5], [6], [7], [8], [9], [10]. In particular, Digital IMC (DIMC) is promising, thanks to the robustness and flexibility of digital circuits [11], [12], [13], [14], [15], [16], [17], [18], [19]. As a result, recent focus has shifted from Analog IMC (AIMC) toward DIMC.

Most DIMC chips are designed with static random access memory (SRAM) because of SRAM's compatibility with advanced CMOS technology. In [13], a fully digital SRAM-based DIMC macro is proposed. This design integrates

a NOR gate into every 6T cell to support multiplication within the SRAM array. In [14], a DIMC macro with a customized 12T cell is proposed to support simultaneous SRAM read/write operations. These designs achieve high throughput and fairly high area and energy efficiency. In [16], [21], [27], data-related techniques are proposed to further increase the macro-level energy efficiency. These designs exploit sparsity present in AI models, and customize the DIMC macros to support this.

With the advances in DIMC macro design, complete integrated systems-on-chip (SoCs) are implemented to demonstrate the capabilities of DIMC at the system level. Axelera integrates four DIMC cores on an SoC [28], capable of performing a 512×512 8b matrix multiplication in a bit-serial fashion, with four weight sets cached locally in the DIMC memory array. The chip achieves 15 TOPs/W for 8b MAC operations at the macro level, while the system-level energy efficiency is not revealed. Similarly, MediaTek integrates 12 convolutional DIMC macros on a highly customized SoC [23]. Each DIMC macro consists of $18 \times 8 \times 72$ 12b memory cells, and is capable of performing 8×72 12b MAC operations. They achieved 23.2 TOPs/W for 12b MAC under Convolution neural network (CNN)-based workloads for super-resolution and noise-reduction applications.

It is important to realize that achieving high system-level energy efficiency in DIMC SoCs relies heavily on the ability to store full weight matrices on-chip. Fig. 1 presents a comparison between the digital PE array and the DIMC array. In the digital PE array, local registers are used to temporarily store weight data, creating bottlenecks due to the high energy demands of weight SRAM access and the excessive number of required registers. DIMC overcomes both issues by providing local, high-density weight storage. As noted in [23], this approach results in a direct power saving of 28% compared to traditional digital PE arrays. However, if local weight storage cannot accommodate the entire workload, data movement between the weight SRAM and the DIMC is still necessary, negating the energy benefits offered by the DIMC array.

In other words, to fully leverage the benefits of the DIMC array, sufficient DIMC memory capacity is required to enable temporal reuse of the model's weights. Limited DIMC memory reduces the number of neural network layers that can be stored,

Received 19 November 2024; revised 21 February 2025; accepted 25 March 2025. Date of publication 16 April 2025; date of current version 1 July 2025. This article was approved by Associate Editor Mahmut Ersin Sinangil. (Corresponding author: Weijie Jiang.)

Weijie Jiang, Marian Verhelst, and Wim Dehaene are with the Department of Electrical Engineering, KU Leuven, 3001 Leuven, Belgium (e-mail: weijie.jiang@kuleuven.be; wim.dehaene@kuleuven.be).

Cédric Caron is with Sony Europe Ltd., KT13 0XW Weybridge, U.K.

Prabhat Avasare and Marc Pauwels are with Sony DepthSensing Solutions, 1160 Brussels, Belgium.

Color versions of one or more figures in this article are available at <https://doi.org/10.1109/JSSC.2025.3557967>.

Digital Object Identifier 10.1109/JSSC.2025.3557967

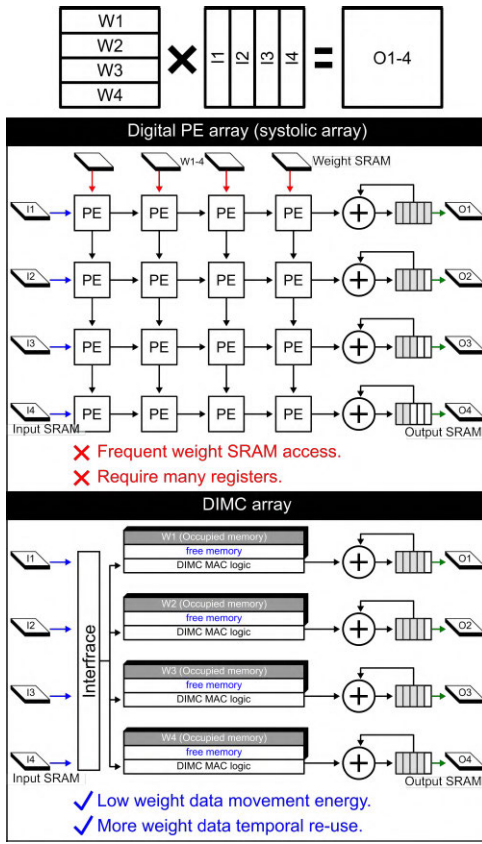


Fig. 1. Comparison between digital PE array and DIMC array.

thereby diminishing energy savings. However, this should not come at the expense of macro-level MAC energy efficiency, as this has a direct impact on overall system-level energy efficiency. In fact, both of them are keys to the system-level energy efficiency.

- 1) Macro-level area utilization efficiency.
- 2) Macro-level MAC energy efficiency.

While macro-level MAC energy efficiency is represented by macro-level TOPs/W, area utilization efficiency at the macro level is captured by a figure of merit (FoM) that combines memory density (bytes per mm^2) and MAC density (MAC units per mm^2). In this article, the FoM is defined as $(\text{MB}/\text{mm}^2) \times (\#\text{MAC}/\text{mm}^2)$.

Moreover, system-level energy efficiency also depends on the DIMC array's average utilization across model layers. While peak macro-level efficiency is high, under-utilization in certain layers can lower overall energy efficiency [31]. Supporting all mappings eliminates under-utilization but increases chip area, reducing area efficiency. Thus, system-level comparisons must evaluate both macro-level area and energy efficiency metrics.

This work presents the HUNBN SoC, an optimized solution designed for edge CNN applications, incorporating four key techniques at the macro and SoC levels to enhance overall system performance.

- 1) *Memory Capacity Optimization*: A foundry 6T bit-cell array is employed for weight storage, significantly boosting system-level memory density to $508 \text{ kB}/\text{mm}^2$.

- 2) *Macro Energy Efficiency Enhancement*: A split-MAC workflow, combined with synchronous DIMC operation, is implemented at the macro level, achieving an energy efficiency of $124 \text{ TOPs}/\text{W}$.
- 3) *Supporting various types of CNN dataflow*: The MAC array is designed with an imbalanced input-to-output dimension ($32 \times 3 \times 3$ to 32) to accommodate commonly used CNN layers. The array can also be reconfigured to support 4×4 transposed convolution for specific tasks.
- 4) *Optimization for Feature Map Data Structure and Data Movement*: The system employs a linear list data structure to represent feature maps and uses a sliding window technique to minimize input/output data movement, further optimizing performance.

The remainder of this article is organized as follows. Section II provides an overview of the SoC components. Section III presents the design details of the DIMC macro. Section IV explores data movement optimization techniques at the SoC level. Section V discusses the measurement results under various conditions, followed by the conclusion.

II. SYSTEM OVERVIEW

The HUNBN SoC, depicted in Fig. 2, comprises a general-purpose RISC-V core, a DIMC cluster, and a multi-bank input/output SRAM data memory. The input/output SRAM is divided into three sections, labeled P1–P3, each containing three banks of 65 kB SRAM. This configuration is specifically optimized for common convolutional filters, as will be discussed later. The input/output SRAM can be accessed either via the AXI bus or directly from the DIMC array. The AXI bus utilizes a lightweight read/write interface designed for low-speed data transfers from the RISC-V core or off-chip sources. In contrast, the DIMC array employs three high-bandwidth stream buses to handle input activation, partial sums, and output activation efficiently, facilitating direct data transfer to and from the SRAM. The DIMC weight data are directly updated via the AXI bus. While the speed and bandwidth of DIMC weight writing are limited, this process is assumed to be a one-time cost, which does not affect performance during runtime.

The DIMC cluster serves as a convolutional layer accelerator with user-defined parameters. The input feature map size, the number of input/output channels, the starting address for the input/output data in SRAM, the weights in DIMC, and the type of convolution kernel (standard or transposed) can all be configured by the RISC-V host core. When properly configured, a CNN layer can be executed on a configurable subset of the 384 customized DIMC macros. These macros are organized into 32 groups, each consisting of 12 DIMC macros. All 32 groups share the same input activation vector, while each group is equipped with an independent post-processing module to handle the results produced by the DIMC macros.

Within each group, every DIMC macro features an SRAM bit-cell array for weight storage and a local MAC unit to perform bit-level multiply-accumulate operations. In the post-processing module, the outputs from all macros within a group are accumulated using two summation stages. The first-stage

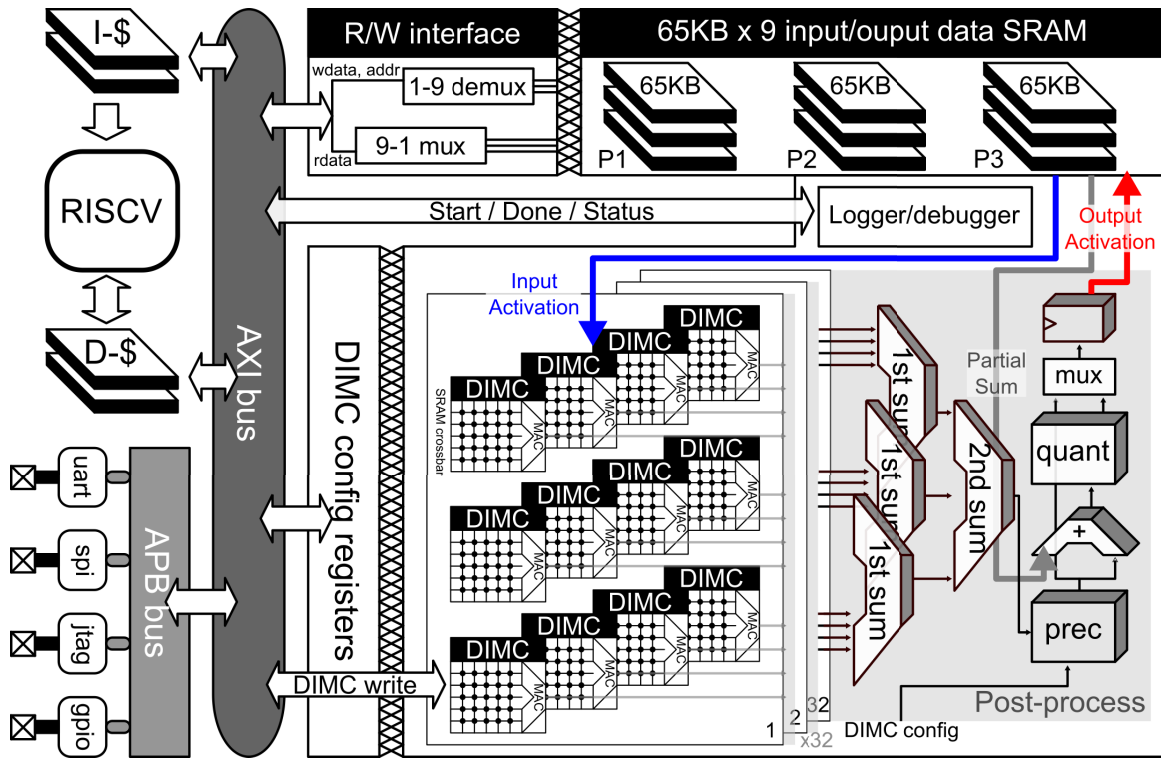


Fig. 2. Architecture of HUNBN SoC.

summation module processes the bit-level MAC results from each DIMC macro by shifting and adding them to form a product-level summation of 4-bit weights and activations. These product-level summations are then accumulated in packs of four. Within each DIMC group, the second-stage summation module then combines the three sums produced by the first stage. The output of the second stage is an 18-bit full-precision value, representing the summation of 288 (24 from DIMC and 12 from post-processing) 4-bit by 4-bit products. It can be either quantized into a 12-bit partial sum, accumulated with a previous 12-bit partial sum and quantized into a 4-bit output, or directly quantized into a 4-bit output. The partial sum or output activation is directly stored in the data SRAM. Additionally, a logger/debugger unit is integrated into the cluster to monitor and control the behavior of the DIMC cluster, ensuring accurate operation and facilitating debugging. Details of each module will be discussed in Sections III and IV.

III. PROPOSED DIMC MACRO

A. DIMC Macro Memory

The proposed DIMC macro utilizes a 6T SRAM bit-cell array, consisting of 256×96 pushed-rule 6T bit cells. This configuration allows the memory to store up to 256×24 4-bit weights. The macro features a standard SRAM access pattern, delivering 24 4-bit weights per cycle in parallel. It then computes the dot product between the 24-element, 4-bit weight vector and the input vector. The SoC, integrated with 384 macros, provides a total storage capacity of 2.36 M 4-bit weights, sufficient to store entire models for most edge CNN applications. The memory is supported by a conventional

SRAM peripheral, where the write peripheral handles memory cell updates and pre-charges the bitline (BL) prior to read operations. A pre-charge signal, Φ , orchestrates the timing of memory operations. Address decoding is managed by a gray-code style decoder that converts an address into a specific word line (WL). By leveraging the minimal transition property of gray code, the area of the decoder is optimized, resulting in improved area efficiency. The timing control circuit design is simplified by exploring synchronous operations. An area breakdown for the proposed DIMC macro is given in Fig. 3. The remainder of this section discusses each block in further detail.

1) *Write/Read Peripheral*: Unlike traditional SRAM, the write/read peripheral in DIMC must be minimized. This is because DIMC typically employs a very high BL to WL ratio to enhance throughput. However, this strategy significantly increases the replication of the write peripheral circuitry. Consequently, even a small reduction or increase in the area of the write peripheral has a substantial impact on the overall area efficiency of the macro. In this work, we explore two key differences between normal SRAM and DIMC.

- 1) DIMC typically utilizes fewer WLs, resulting in significantly lower BL parasitic compared to traditional SRAM. Consequently, full-swing reading on the BL in DIMC is more energy efficient than in traditional SRAM.
- 2) For models that can be fully stored within the DIMC, write operations are only required during the boot process. This significantly reduces the need for write-assist circuitry.

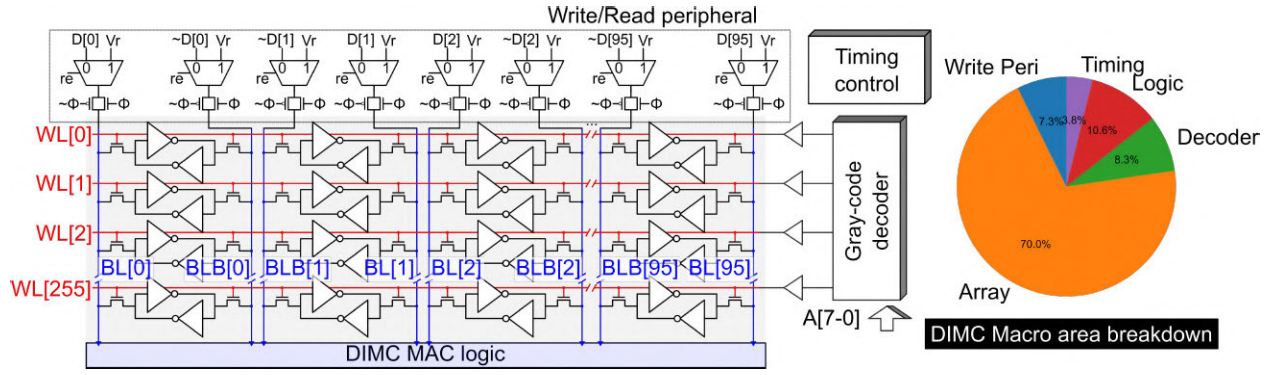


Fig. 3. Reviewer 2.1 circuit details of DIMC macro memory.

As illustrated in Fig. 3, the write/read peripheral functions as a two-way multiplexer, controlled by a read enable signal (re) that dictates the DIMC operation mode. When re is set to 0, 96 different data inputs ($D[0-95]$) are written into a selected WL. Conversely, when re is set to 1, a pre-charge voltage (Vr) is applied to the BL. The value of Vr is adaptable; for example, a lower Vr , such as 75% of the nominal V_{dd} , can improve reading speed and reduce energy consumption. However, this also decreases cell robustness and limits the maximum voltage that can be applied to the DIMC MAC logic. In all modes, changes occur only when the control signal Φ is active (high).

2) *Gray-Code Style Decoder*: To further optimize the area efficiency of DIMC circuitry, this work proposes a gray-code style decoder to improve the decoder routing efficiency. As illustrated in Fig. 4, the 8-bit address $A[7-0]$ is partitioned into three segments: $A[7-6]$, $A[5-3]$, and $A[2-0]$. A binary-to-thermometer conversion module generates the enable (ENi) signals to select one of the four blocks, each containing 64 addresses. Within each block, the addresses are further subdivided into eight smaller tiles, selected by decoding $A[5-3]$. Within a tile, the $A[0]$ bit functions as a special address signal that persists for half of the duty cycle. This bit is also encoded in gray code, ensuring only a single bit change between consecutive WL drivers. As illustrated in Fig. 4, the proposed gray-code decoder offers more opportunities for gate poly contact sharing in the NAND gates compared to the conventional binary-code decoder. This reduces layout complexity. Moreover, it reduces the switching activities on the WL for linear address fetching. On average, the proposed decoder saves 50% dynamic energy compared to the baseline binary number decoder, thanks to the minimal bit flipping.

B. DIMC MAC Logic

Fig. 5 illustrates the DIMC MAC logic. In each clock cycle, 24 4-bit weights are read from the DIMC memory, denoted as W_0 to W_{23} . Weight bits with the same significance are grouped together (four groups of 24 bits each), and receive the same input activation for subsequent multiplication. Each cycle, each of these weight bit groups, is, moreover, fed with 2 bits of the input activation to perform a 1-bit by 2-bit multiplication. To reduce the capacitive load on each line, both BL and BLB lines are utilized in this 1-bit by 2-bit multiplication, each toward a different bit of the 2-bit mul-

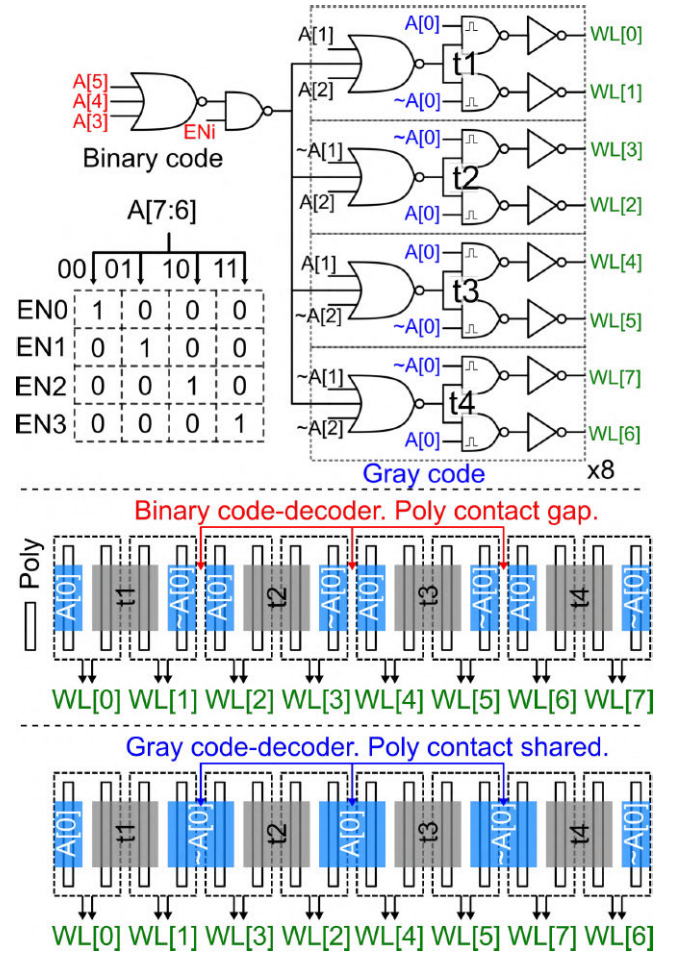


Fig. 4. Proposed gray-scale decoder.

tiplication result, respectively. The macro as such multiplies 24 2-bit input activations, labeled I_0 to I_{23} , with W_0 to W_{23} . Since the weight and its inverse are read on BL and BLB, the two bits of input activations use different signs to ensure correct bit-level multiplication. This operation can be repeated in a 2-bit serial way across subsequent clock cycles for higher input precision, e.g., enabling a complete 4-bit by 4-bit MAC operation in just two clock cycles, or twice as fast as a traditional bit-serial operation. The multiplication results are further accumulated using a customized adder tree, which operates in a low-voltage domain. In other words, the

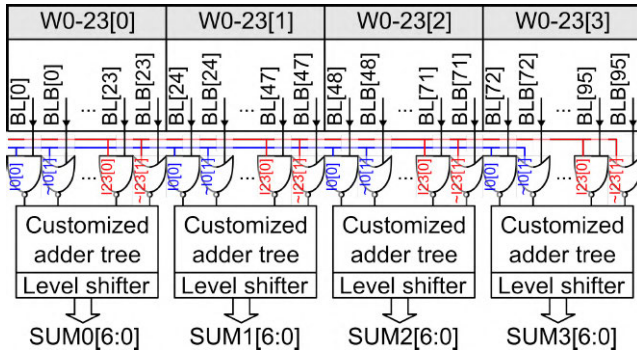


Fig. 5. Detail of DIMC MAC logic.

DIMC MAC circuits that are connected to the BL/BLB are all supplied by a low voltage supply. This configuration enables the DIMC to function at a lower V_r during reading operations, minimizing leakage issues. The summed results (SUM0 to SUM3) are finally restored to standard digital swing levels using static-level shifters before being processed further in the synthesized digital domain. These level shifters increase the macro area by only 2%.

C. DIMC Synchronous Operation Timing Diagram

To further enhance macro-level energy efficiency, we implement synchronous operations for the DIMC, reducing the energy required for data reading within the DIMC. The timing diagram of the DIMC macro is depicted in Fig. 6, highlighting three primary operations: write, read/compute, and refresh. During the weight write operation, weight data (D) is written to the word addressed by the address A . Since the write process inherently disrupts the memory state, the overlap between the control signal Φ in Fig. 3 and the WL is not a concern, enabling continuous writes over consecutive clock cycles. In the read/compute operation, the macro reads a word from memory and performs the MAC calculation with the 2-bit input activation (I). In this mode, overlap between Φ and WL is not allowed, as it could lead to destructive reading. While traditional SRAM employs delay lines to address these issues, robust and variation-resilient delay lines are both area- and power-intensive, creating a significant bottleneck for DIMC arrays with numerous macros. In this work, synchronous reading/computing is proposed to avoid this overlap concern. The idea is to spread the Φ and WL pulse over two different clock cycles in a synchronous way. A pre-charge enable (pe) signal proceeds each read operation, generating a Φ pulse to pre-charge the BL. A read enable (re) signal follows right after, reading the selected word onto the BL. After a read operation, numerous compute operations can follow. Each SUM is the MAC result of the BL weight data and a different input (I). Since BLs are floating nodes during the compute operation, a refresh pulse needs to be inserted on the WL after a while to recover the values on the BL to full swing. This mechanism does not affect throughput, as the BL values remain valid and ce stays high during refresh. It also has minimal impact on energy consumption, since the BL values are already close to the read values, resulting in only a small BL voltage

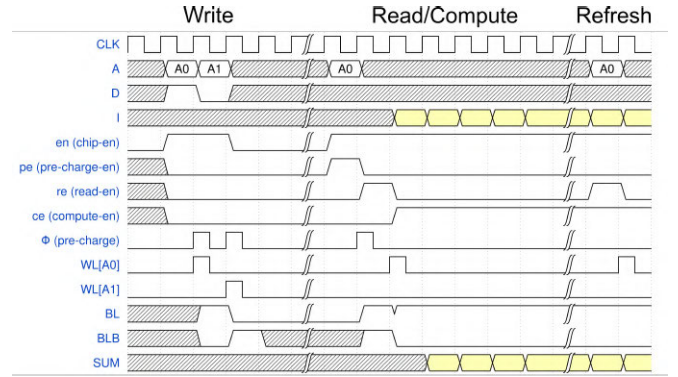


Fig. 6. Reviewer 1.4 timing diagram of the proposed DIMC macro.

change. However, this mechanism begins to degrade when there is reduced temporal reuse of weights. The impact of this will be examined in Section V by comparing among different workloads.

The adoption of synchronous operation significantly reduces the critical path delay during the read operation. Additionally, since the DIMC MAC logic is no longer part of the critical path, its power supply can be lowered to enhance energy efficiency. For subsequent measurements, both the DIMC MAC logic power supply and the pre-charge voltage (V_r) are set to the minimum voltage level at which correct computation results are still reliably obtained.

IV. PROPOSED SOC OPTIMIZATION TECHNIQUES

A. Split MAC Workflow

To fully exploit the efficiency and density of DIMC while minimizing manual circuit design effort, this work introduces a split MAC workflow. Fig. 7 compares the traditional DIMC MAC workflow with the proposed split MAC workflow. In the conventional DIMC approach, the manual design effort is concentrated in two areas: the multiplier, which scales with the precision of the weights and inputs, and the adder tree, whose complexity is proportional to the product of the precision and the logarithm of the number of inputs. This is because each stage of the adder tree requires a different bit-length adder, necessitating unique designs for each stage. In contrast, the proposed split MAC workflow integrates the early stages of the adder tree into the manually designed multiplication stage, while it pushes the rest of the adder tree into the synthesized logic. This approach offers two key advantages.

- 1) The most repeated components, such as early-stage accumulation and bit-level multiplication, are fully optimized using a customized circuit design flow within DIMC. Compared to the baseline circuit implemented using a standard digital flow, a 20% area reduction is achieved. This gain results from customized logic gate height and utilizing the lowest metal layer, which digital tools cannot fully exploit. This part is treated as a hard macro in HDL.
- 2) The more complex yet less reused components, including the partial product combiner and late-stage accumulation, are implemented at the RTL level using a standard digital design flow.

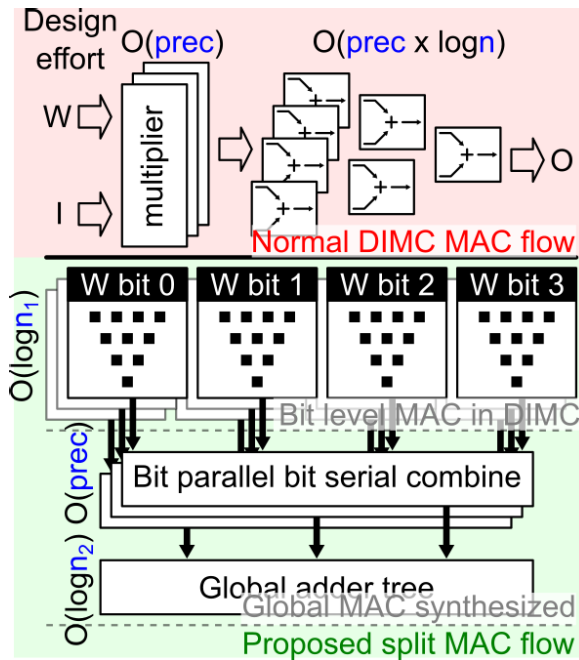


Fig. 7. Comparison between normal DIMC MAC flow and proposed split MAC flow.

This strategy leverages the strengths of DIMC where it yields the most, while significantly reducing the overall design time.

B. CNN Dataflow on SoC

CNNs are widely used AI models for visual tasks such as image recognition and segmentation. Fig. 8 illustrates the computation flow of a CNN layer. It processes a 3-D input tensor and a 4-D kernel tensor, performing matrix multiplication over each window of the input feature map to produce a 3-D output tensor. Unlike traditional matrix multiplication, adjacent output pixels share many common input pixels, which allows for optimized data reuse and reduced data movement. The parameters shown in Fig. 8 can vary depending on the type of convolution layer used. The HUNBN chip supports both normal CNN layers and transposed (up-sampling) CNN layers, with the primary difference being the input fetch pattern. In normal convolution, all pixels within the window are convolved with the kernel, while in transposed convolution, zeros are inserted to up-sample the input feature map. The HUNBN chip also supports a wide range of layer dimensions (IX, IY, C, K, FX, and FY) with varying temporal execution loops. Four representative cases are illustrated below.

- 1) *Case 1*: $FX = 3$, $FY = 3$, $C = 32$, and $K = 32$. This configuration serves as the cornerstone of all spatial unrolling. As illustrated in Fig. 9(a), the 384 macros are arranged in a $3 \times 4 \times 32$ constellation, where the $FY = 3$, $C1 = 4$, and $K = 32$ loops are mapped at the cluster level. The inner $C2$ and FX loops of the CNN are mapped within each macro. Since the macro performs 24 vector multiplications, up to $FX = 3$ and $C = 8$ can be supported within a single macro. The FY , K , and $C1$ loops are computed in parallel in the cluster. The OX and OY loops, finally, are computed in the temporal

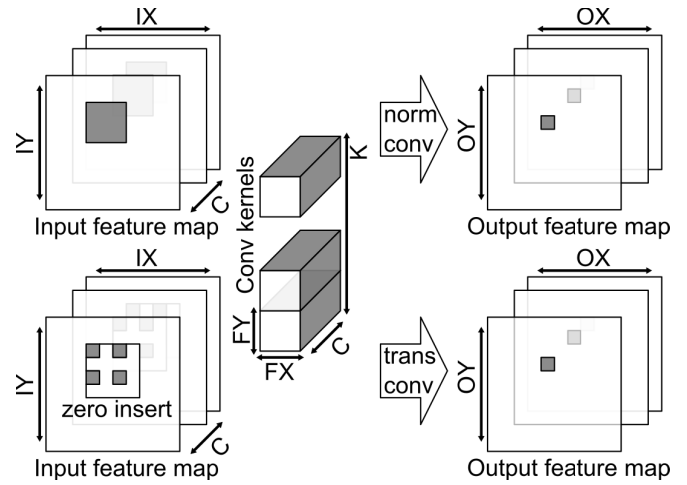


Fig. 8. Computation of a general CNN layer.

domain, e.g., stored in input/output SRAM. This setup is optimal as it efficiently utilizes exactly one DIMC address. It also enables weight reuse at a single DIMC address throughout the entire layer computation process, maximizing the macro-level energy efficiency.

- 2) *Case 2*: $FX = 3$, $FY = 3$, $C > 32$, and $K > 32$. This configuration extends Case 1. Here, the CNN layer requires more than one row of each DIMC macro, leading to storage across consecutive DIMC addresses, and an iteration across subsets of C and K in a temporal fashion. An example case with $C = 64$ and $K = 64$ is illustrated in Fig. 9(b). The layer is computed using a weight-stationary dataflow, where the $K0$ and $C0$ loops are placed at the outer temporal loop, followed by OX and OY . This approach offers two key benefits: 1) *Reduced Weight Reading Frequency*: weight-stationary dataflow minimizes the need for frequent weight reads, thus lowering the penalty from synchronous DIMC operations and 2) *Improved Input Data Reuse*: by leveraging data locality in the FX dimension through a sliding window approach, input data reuse is enhanced.
- 3) *Case 3*: $FX > 3$, $FY > 3$, $C = 32$, and $K = 32$. This configuration represents scenarios where the kernel size exceeds the baseline dimensions of $FX = FY = 3$. An example with $FX = FY = 5$ is illustrated in Fig. 9(c). In this setup, the kernel is divided into subkernels by segmenting the dimension by 3 and rounding up to the nearest integer. The kernel is then split into four parts: the upper section containing a complete 3×3 and a 2×3 subkernel, and the lower section contains a 3×2 and a 2×2 subkernel part. These subkernels are stored in consecutive DIMC addresses. To prevent erroneous computations, zeros are inserted into the memory for the smaller parts. Macros within a cluster are categorized into highly utilized macros (shown in gray) and less utilized macros (shown in red). Due to the suboptimal mapping of the kernel, the performance in this case is lower compared to Cases 1 and 2.
- 4) *Case 4*: $FX = 4$, $FY = 4$, $C = 18$, and $K = 32$. This configuration represents a specific scenario where the layer

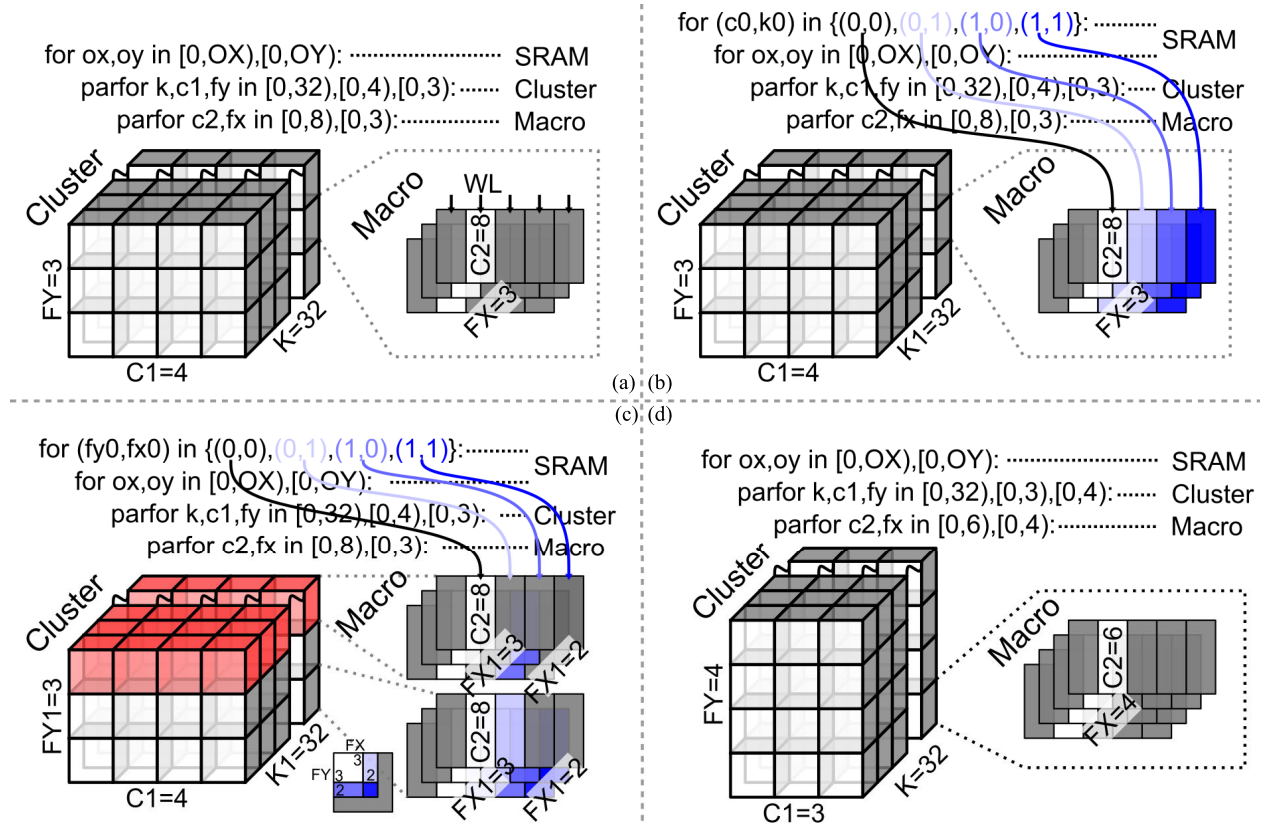


Fig. 9. Illustration of CNN spatial mappings: Cases 1–4 are demonstrated in (a)–(d).

is a transposed convolution layer. The DIMC macros are organized into a $4 \times 3 \times 32$ cluster, as shown in Fig. 9(d). The loop parameters $FY = 4$, $C1 = 3$, and $K = 32$ are mapped to the cluster, while inside the macro, $FX = 4$ and $C2 = 6$ are mapped to the weight vector at a single address. This spatial mapping fully utilizes DIMC's memory and MAC resources. However, on HUNBN, the input channel count is usually a multiple of 32, which misaligns with the optimal configuration ($C = 18$) for the transposed convolution layer and complicates input/output data movement management. HUNBN provides special support for these irregular cases, which will be discussed in Section IV-C.

C. Data Movement Optimization for Input Activation

To further enhance system-level energy efficiency, the input activation data movement from the SRAM to DIMC cluster is optimized toward the DIMC cluster architecture. Fig. 10 illustrates the memory representation of an example input feature map. The 3-D tensor is decomposed into four hierarchical levels to optimize memory storage and access.

At the first level, pixels across different channels are grouped into units called PixelPacks, with each pack containing pixels from 32 different channels. For layers with more than 32 channels, multiple PixelPacks are used to store the pixels at a given X/Y -coordinate. Since each pixel is a 4-bit value, 32 pixels require 128 bits of storage, which aligns with the word length of the input/output SRAM. As a result, the PixelPack serves as the smallest storage unit for the feature map.

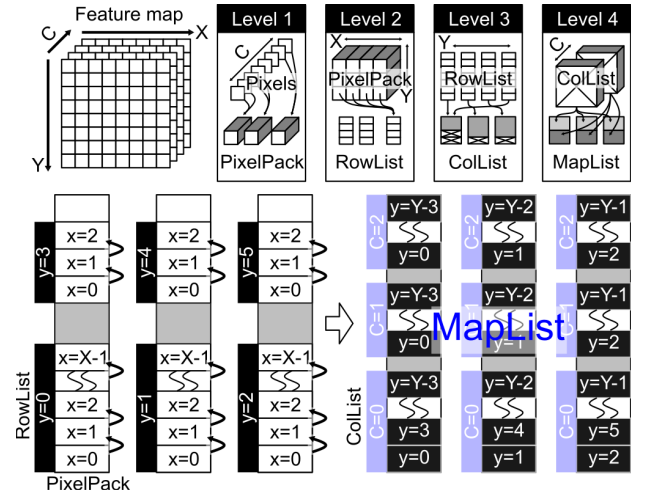


Fig. 10. Memory representation of an example feature map.

At the second level, PixelPacks corresponding to the same row (y -dimension) are arranged into linear structures called RowLists. A RowList contains the starting address and x -length of the feature map to locate the associated PixelPacks. For most CNN layers, this linear fetching mechanism is sufficient, significantly simplifying memory management.

At the third level, these RowLists are further organized into ColLists. The ColLists contain every pixel of the feature maps in groups of 32 channels, making them the smallest operable unit for layer-wise scheduling. For example, if a feature map contains 64 input channels, it requires two ColLists for storage.

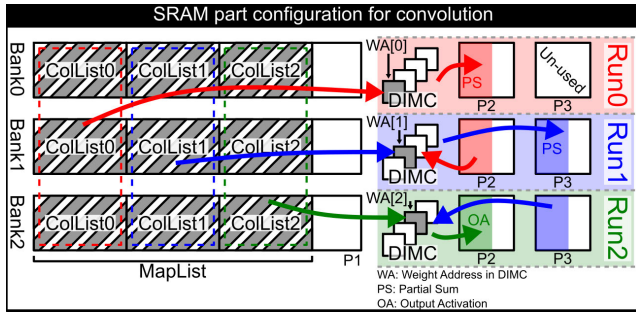


Fig. 11. Illustration of an example SRAM part configuration.

To avoid memory access conflicts, each ColList distributes the RowLists across three separate SRAM banks, where the location is determined by the RowList's modulo-3 congruence. With this configuration, three consecutive rows are distributed across different banks, ensuring single-bank access per cycle for an $FX = FY = 3$ layer.

At the fourth level, ColLists that originate from different input channels are distributed across the SRAM. The complete set of ColLists is referred to as the MapList, which represents the entire feature map.

Before the convolution begins, the SRAM parts must be configured. Fig. 11 shows an example of part assignment for a CNN layer with $C = 96$. In this setup, P1 serves as the input activation supplier, P2 as the first partial sum buffer, and P3 as the second partial sum buffer. Given $C = 96$, the input activation and weight tensor require three ColLists and three DIMC addresses. Consequently, three cluster runs are needed to compute the output activation. During these runs, P2 and P3 alternate roles as partial sum buffers. Data movement is optimized differently for normal 3×3 convolution compared to 4×4 transposed convolution.

1) *Normal Convolution*: Fig. 12 illustrates the data movement in a typical convolution operation. For simplicity, only run 1 and a portion of run 2 are discussed. Fig. 12(a)–(c) depicts the data movement process for run 1.

During run 1, the base address is initially set to 0, corresponding to the starting address of the first RowList in ColList 1. An x -pointer is used to address the PixelPacks in RowList 0, RowList 1, and RowList 2, with an offset based on the current x -coordinate. The PixelPacks are cached locally in a three-depth register, which supplies the complete dataset needed for one convolution. Concurrently, the same x -pointer is used to fetch the previous partial sum from P2 and store the current partial sum to P3. Upon completion of the first row, the computation proceeds to the second row. As shown in Fig. 12(b), RowLists RL1–RL3 are required. The ColList layout ensures that no bank conflicts arise. RL1 and RL2 continue to use base address 0, while RL3's base address is set to $0 + IX$ to correctly locate RL3 in bank0. Similarly, the partial sum is stored with an offset of $0 + IX$ to prevent row data overwrite. This setup allows the computation of the third row to proceed without bank conflict as well, as shown in Fig. 12(c).

A portion of run 2 is illustrated in Fig. 12(d). The input activation fetch pattern follows the same rules as in run1, with the primary differences occurring in the output. First, while

run 1 stores the partial sum in P3, run 2 retrieves the partial sum from P3 instead of P2. Second, partial sums are stored using 12 bits, which do not fit within a standard PixelPack. To address this, three PixelPacks across three banks at the same address are combined to form an extended PixelPack for partial sum storage. In contrast, the output activation requires only 4 bits and thus fits within a single PixelPack in one bank. In this case, the x -pointer is set to access the correct bank according to the previously discussed rules, enabling seamless integration for the computation of the next layer.

2) *Transpose Convolution With $FX = FY = 4$* : Special support is provided for transpose convolution layers with $FX = FY = 4$, introducing two key differences: 1) the input feature map is up-sampled by inserting zeros, requiring a more complex line-fetching mechanism and 2) the maximum input channels per cycle are reduced to 18, which can now span across two PixelPacks, adding to the control overhead. Fig. 13(a) illustrates an example of data movement. In transposed convolution, zeros are inserted in one column, reducing the required input data bandwidth. However, due to the mismatch between the PixelPack C dimension and the spatial unrolling, two PixelPack accesses may be required to retrieve the complete set of input activations. Fortunately, with the insertion of zeros, the frequency of new input requests is effectively halved compared to standard convolution, as shown in Fig. 13(c). This allows a data look-ahead mechanism to keep the DIMC operating at the same pace as in normal convolution. An example of this is shown in Fig. 13(d): at time $t = n$, input data from the first portion of the input channels for the next pixel are accessed from the appropriate PixelPack in ColList1. At $t = n + 1$, when new pixel data is required, the second portion is accessed from the correct PixelPack in ColList2. Altogether, there are 16 possible fine-grain PixelPack fetching patterns, as illustrated in Fig. 13(e). Such mechanism enables full-speed execution of $FX = FY = 4$ transpose convolution.

D. Precision Control for the Partial Sum

Since a large number of values can be accumulated together, special attention is given to handling potential overflow situations. Fig. 14 illustrates the four supported modes for managing this. While the partial sums stored in SRAM are limited to 12 bits, 18 bits are allocated for the original data. The user can select the dynamic range based on the accumulation depth of each layer. For early layers with fewer input channels, the first 12 bits can be used for partial sums. In deeper layers with a larger number of input channels, bits 4 through 16 can be selected to accommodate larger sums. This approach sacrifices precision in the LSB but effectively prevents overflow. In all modes, the final outputs are quantized to 4 bits whose dynamic range is compressed by a floating point number for scaling. Since layers with more input channels are quantized using smaller scaling factors, the impact of LSB differences on the final output is less significant.

V. MEASUREMENT RESULTS

A. Basic Measurement

This chip is fabricated in TSMC 16-nm FinFET technology. The chip is designed under 200-MHz sign-off speed

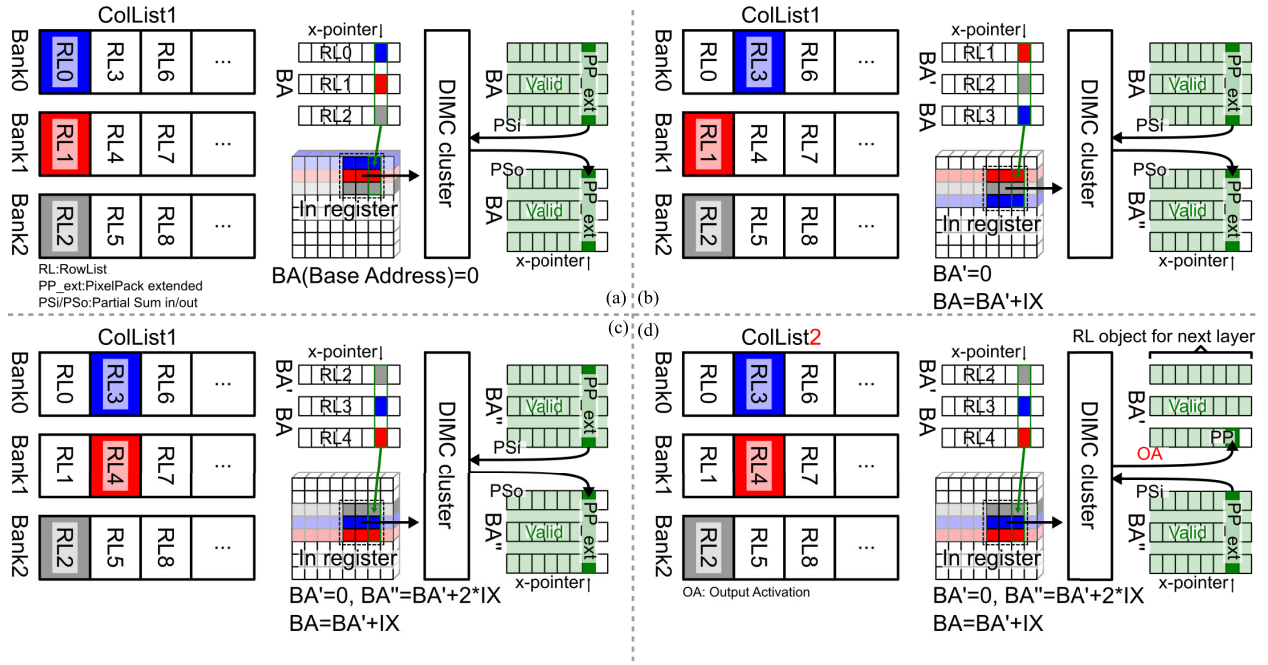


Fig. 12. Data movement for normal convolution: (a)–(c) illustrate the process of run 1, while (d) shows the differences between run 1 and run 2.

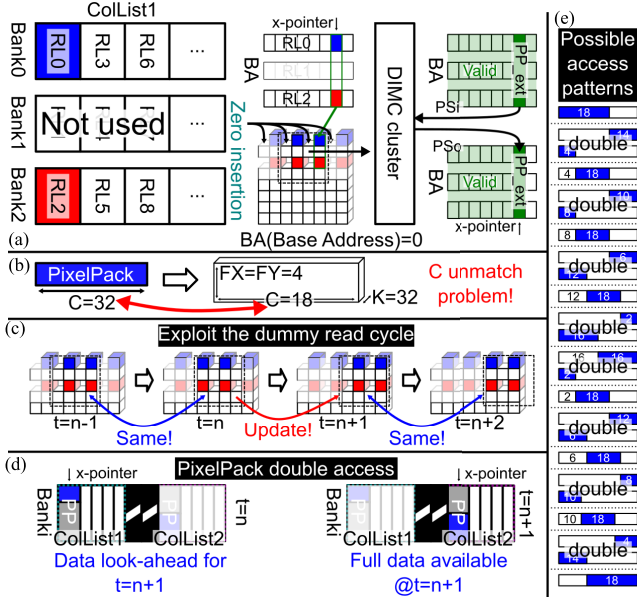


Fig. 13. Data movement for $FX = FY = 4$ transpose convolution. (a) Shows data movement of an example transposed convolution. (b) Illustrates the mismatch problem on the C dimension. (c) Explains the existence of dummy read cycle due to the insertion of zeros. (d) Illustrates the data look-ahead mechanism. (e) Gives all possible PixelPack fetching pattern for the data look-ahead mechanism.

to save power for edge applications. Fig. 15 shows the measurement setup and a chip photograph. The total core area is 1.9×1.9 mm. Fig. 16 shows the measured Shmoo plot of four samples. The definition of failure is the occurrence of the first error in any MAC results of all macros on one sample. All samples can run at maximal 165 MHz at 0.88 V core voltage. Fig. 17 shows the frequency versus efficiency plot at both SoC level and macro level. The SoC-level energy efficiency

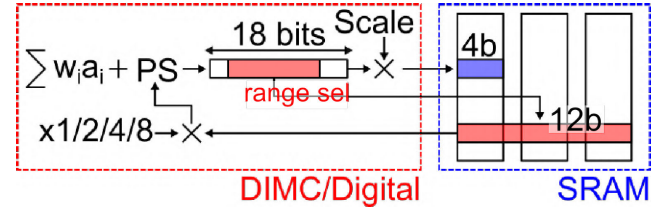


Fig. 14. Precision control for large accumulation.

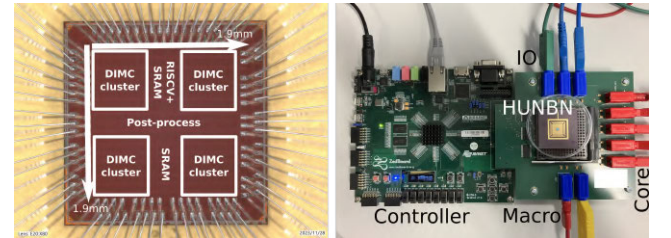


Fig. 15. Measurement setup and chip photograph.

includes every energy source on-chip, excluding the IO power domain. This work achieves 24.28 TOPs/W at system level and 126.3 TOPs/W at macro level for purely random test data. Table I presents a comparison with state-of-the-art solutions. This work offers the highest on-chip IMC capacity, with a total memory density of 508 kB/mm². Using an FoM defined as (MB/mm²) $\times$ (#MACs/mm²), this design achieves 0.64 K, representing a 20% improvement over the best competitor. The FoM indicates that our chip is optimally suited for edge applications that prohibit off-chip DRAM access. Its superior macro-level area utilization efficiency reduces the required chip area for the same workload compared to other designs, conserving valuable chip space. This area savings can be

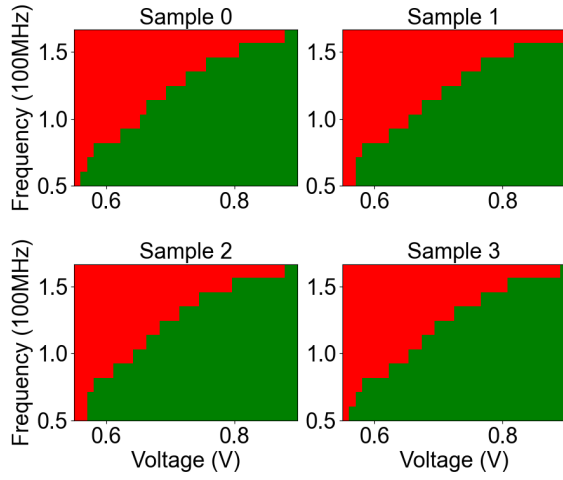


Fig. 16. Shmoo plot for four measured samples.

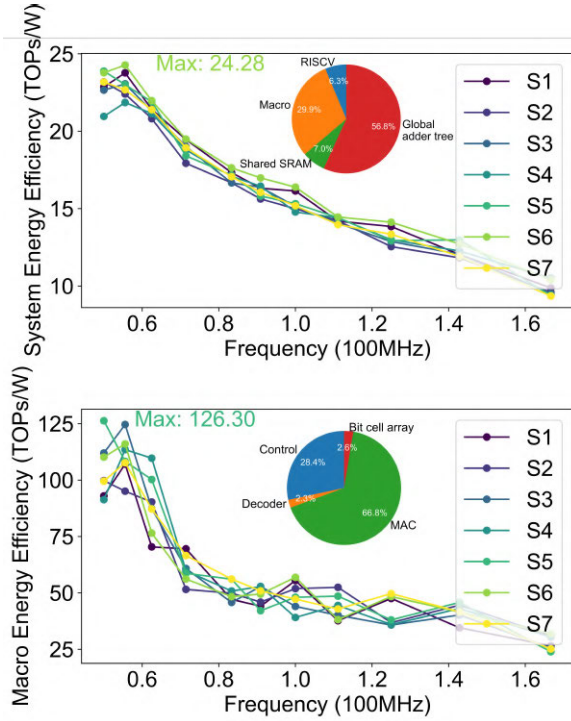


Fig. 17. Energy efficiency of seven measured samples under different frequencies. The input and weight are pure random numbers.

redirected toward manufacturing cost reduction or allocated for other critical functions.

B. Evaluation on Different Layer Type

Finally, HUNBN's flexibility and efficiency are demonstrated by mapping complete end-to-end workloads. Fig. 18 illustrates the equivalent energy efficiency and throughput for convolution operations across varying feature map sizes. The convolution kernel is fixed with dimensions $FX = 3$, $FY = 3$, $C = 32$, and $K = 32$, while both input and weight sparsity levels are maintained at 50%. For small feature map sizes, the SoC experiences reduced energy efficiency and throughput due to frequent line switching and DIMC stalling compared

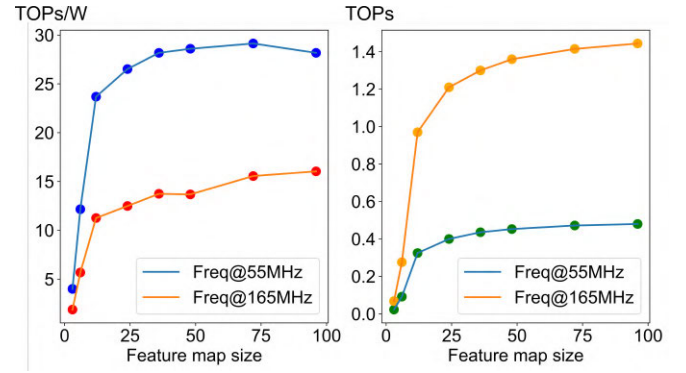
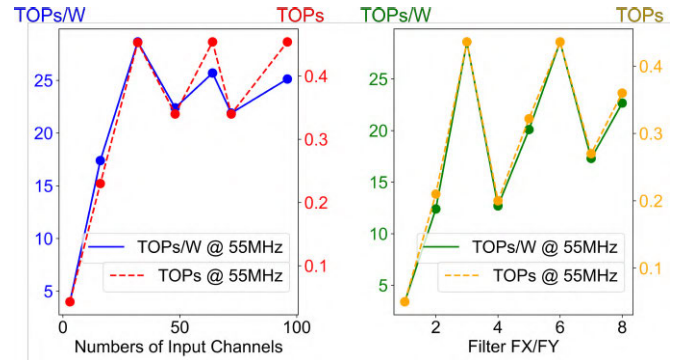
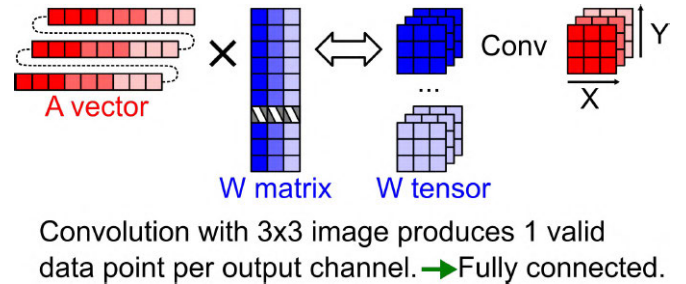


Fig. 18. Energy efficiency and throughput versus different feature map size.

Fig. 19. Energy efficiency and throughput versus different number of input channels and the dimension of filter FX/FY .Fig. 20. Replacing fully connected layer with convolution on 3×3 input feature map.

to large feature map sizes. In this scenario, clock power and static energy consumption dominate the overall power usage. As the feature map size increases, both energy efficiency and throughput show significant improvement. However, this gain begins to plateau around a feature map size of 24, where the control power overhead becomes negligible. The maximum energy efficiency achieved is 29 TOPs/W at 0.42 TOPs. In conclusion, unless the feature map is extremely small, the SoC can operate at its full potential, delivering optimal performance.

Fig. 19 illustrates the performance across different convolution kernels, varying in dimensions FX/FY and input channels C , while keeping the output channel fixed at $K = 32$ for simplicity. A periodic pattern is observed in relation to both the input channel count and the filter size. The SoC operates most efficiently when the input channel count (C) is a multiple of 32, and the filter size (FX/FY) is a multiple

TABLE I
COMPARISON TO SOTA DIMC MACROS AND SoCs

Design Reference	Macro chips			SoC chips			
	[30]	[14]	[16]	[22]	[21]	[20]	<i>This work</i>
Technology	4nm	5nm	28nm	18nm	28nm	55nm	16nm
Technology scale (um ²) ^a	0.0199 ^b	0.021	0.124	0.074 ^c	0.124	0.4	0.074
Core area (mm ²)	0.0172	0.0133	6.69	4.2	4.536	12.87	3.61
IMC capacity	8KB	6.75KB	12KB	128KB	8KB	128KB	1180KB
Total memory (SRAM+IMC)	NA	NA	0.14MB	NA	0.552MB	1.392MB	1.835MB
Total memory density ^d	125KB/mm ²	144KB/mm ²	35KB/mm ²	NA	204KB/mm ²	583KB/mm ²	508KB/mm ²
Total MAC density ^{d,e}	9K/mm ²	21K/mm ²	1.84K/mm ²	NA	1.5K/mm ²	0.86K/mm ²	1.27K/mm ²
Max throughput ^e	3.4TOPs	2.94TOPs	5.36TOPs	NA	1.64TOPs	1.02TOPs	1.53TOPs
SoC efficiency ^e	NA	NA	78TOPs/W ^f	NA	51TOPs/W ^f	5.62TOPs/W	24TOPs/W
FoM ^g	NA	NA	NA	NA	0.3K	0.5K	0.64K

^aTechnology scale is defined by the area of a SRAM bit cell.

^{b,c}4nm bit cell area is missing. 3nm data is used. 18nm bit cell area is missing. 16nm data is used.

^dAll density numbers are scaled to 16nm by the technology scale.

^eScaled to 4b×4b MAC units. All ops are 4b. 1 MAC=2 ops. No sparsity considered.

^fDIMC loading energy overhead not included in the number.

^gFoM is defined as (MB/area)×(#MAC/area). Both area are scaled based on technology scale.

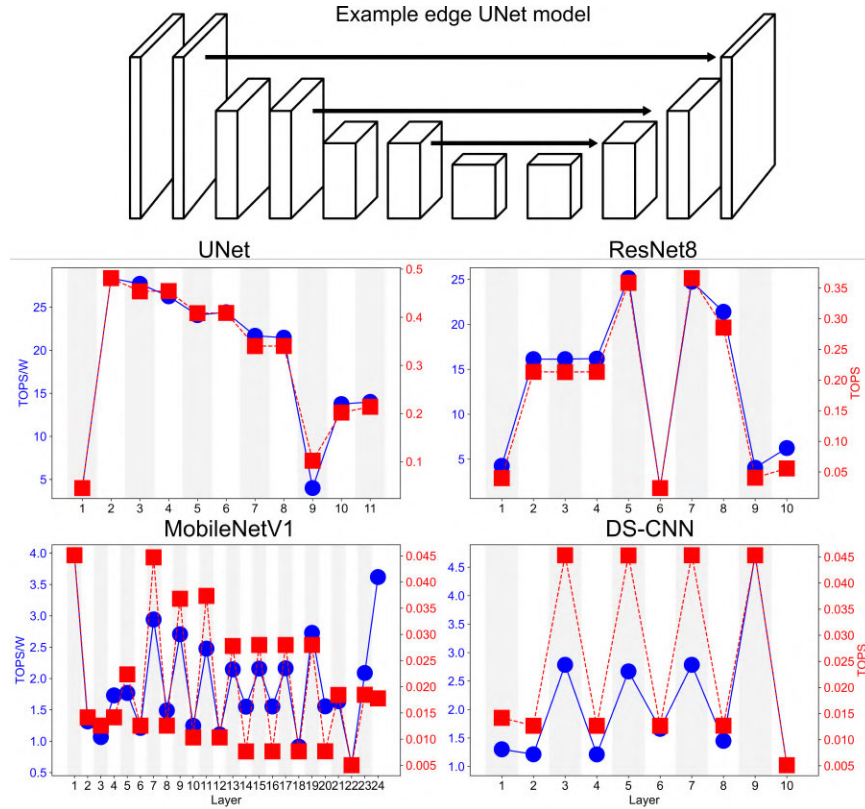


Fig. 21. Performance analysis for U-Net, ResNet8, MobileNetV1, and DS-CNN.

of 3. Notably, the performance significantly degrades for cases like $FX = FY = 1$ (e.g., point-wise convolution), due to severe under-utilization of the hardware resources.

C. Evaluation on Different Workloads

1) *U-Net and ResNet8*: A U-shape network (U-Net) model and ResNet8 have been fully mapped onto the SoC. The U-Net is a customized 4-bit model without significant accuracy drop (<1%) compared to the original 8-bit model. The ResNet8 and other benchmarks are directly taken and quantized to 4b according to [30]. Notably, the fully connected layer at the

end of ResNet8 has been replaced by a convolution layer with dimensions $FY = 3$ and $FX = 3$, applied on a 2-D input with X -dimension = Y -dimension = 3, as Fig. 20 shows. This adjustment gives equivalent computations. Apparently, such approximation results in many stalls in the system due to the limited size of the input feature map. All subsequent fully connected layers in the model are handled similarly to maintain computational efficiency. Fig. 21 provides a visual of the U-Net architecture alongside measurement data for both U-Net and ResNet8. For the U-Net, the long skip connections are efficiently implemented on-chip. The SoC achieves an average system-level performance of 19 TOPS/W and

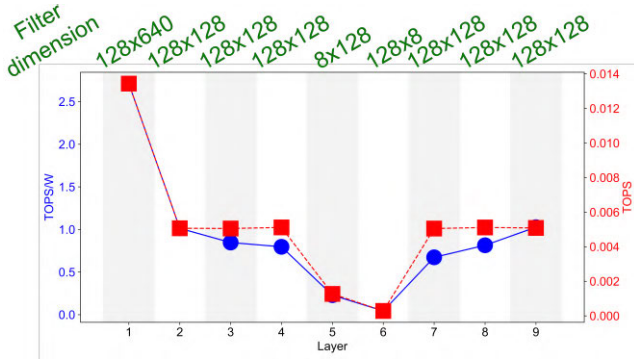


Fig. 22. Performance analysis for deep autoencoder.

0.3 TOPS across all layers, consuming a total of 0.11 mJ per frame at 155 frames/s. For ResNet8, we achieve an average of 13.6 TOPS/W and 0.18 TOPS specifically for the convolution layers, with a total power consumption of 1.5 uJ per frame at 8.44 K · frames/s. The average system-level performance for U-Net is 80% of the peak performance, and for ResNet8 is 57% of the peak performance, demonstrating the benefits of flexible dataflow support.

2) *MobileNetV1 and DS-CNN*: Fig. 21 shows the measurement data for MobileNetV1 [30] and DS-CNN [30]. For MobileNetV1, this work achieves 1.9 TOPs/W and 0.02 TOPs for MobileNetV1, and 3.7 uJ per frame at 2.88 K · frames/s. For DS-CNN, this work achieves 2.08 TOPs/W and 0.025 TOPs, and 5.09 uJ per frame at 2.44 K · frames/s. Unlike U-Net and ResNet8, point-wise and depth-wise filters dominate these models, which lead to under-utilization of the array. This explains the efficiency gap between MobileNetV1/DS-CNN and U-Net/ResNet8.

3) *Deep Autoencoder*: Deep autoencoder [30] is also tested on the SoC. As mentioned, all GEMM operators are replaced with approximate convolution layers. Fig. 22 shows the measurement data for deep autoencoder. This work achieves 0.9 TOPs/W and 5 GOPs, and 0.17 uJ per frame at 30.8 K · frames/s.

VI. CONCLUSION

This work presents a high-density DIMC SoC designed for edge CNN applications. By utilizing pushed-rule memory cells, HUNBN can store the entire edge-based CNN within the DIMC memory, significantly reducing energy costs associated with frequent on-chip weight loading. More importantly, through innovative DIMC operation techniques and a customized dataflow, HUNBN maximizes data reuse in CNNs, achieving exceptional energy efficiency and throughput. As a result, we achieve an SoC energy efficiency of 24 TOPS/W for edge CNN models. Furthermore, the performance is adaptable to other models, thanks to the high area utilization ratio and the built-in dataflow flexibility of the DIMC processor.

REFERENCES

[1] S. Yin, Z. Jiang, J.-S. Seo, and M. Seok, "XNOR-SRAM: In-memory computing SRAM macro for binary/ternary deep neural networks," *IEEE J. Solid-State Circuits*, vol. 55, no. 6, pp. 1733–1743, Jun. 2020.

[2] Z. Jiang, S. Yin, J.-S. Seo, and M. Seok, "C3SRAM: An in-memory-computing SRAM macro based on robust capacitive coupling computing mechanism," *IEEE J. Solid-State Circuits*, vol. 55, no. 7, pp. 1888–1897, Jul. 2020.

[3] A. Biswas and A. P. Chandrakasan, "CONV-SRAM: An energy-efficient SRAM with in-memory dot-product computation for low-power convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 54, no. 1, pp. 217–230, Jan. 2019.

[4] N. Verma et al., "In-memory computing: Advances and prospects," *IEEE Solid-State Circuits Mag.*, vol. 11, no. 3, pp. 43–55, Aug. 2019.

[5] H. Jia et al., "Scalable and programmable neural network inference accelerator based on in-memory computing," *IEEE J. Solid-State Circuits*, vol. 57, no. 1, pp. 198–211, Jan. 2022.

[6] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, "Memory devices and applications for in-memory computing," *Nature Nanotechnol.*, vol. 15, no. 7, pp. 529–544, Jul. 2020.

[7] H. Dbouk, S. K. Gonugondla, C. Sakr, and N. R. Shanbhag, "A 0.44-μJ/dec, 39.9-μs/dec, recurrent attention in-memory processor for keyword spotting," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2234–2244, Jul. 2021.

[8] J.-W. Su et al., "Two-way transpose multibit 6T SRAM computing-in-memory macro for inference-training AI edge chips," *IEEE J. Solid-State Circuits*, vol. 57, no. 2, pp. 609–624, Feb. 2022.

[9] J.-S. Seo et al., "Digital versus analog artificial intelligence accelerators: Advances, trends, and emerging designs," *IEEE Solid State Circuits Mag.*, vol. 14, no. 3, pp. 65–79, May 2022.

[10] B. Zhang et al., "A 177 TOPS/W, capacitor-based in-memory computing SRAM macro with stepwise-charging/discharging DACs and sparsity-optimized bitcells for 4-bit deep convolutional neural networks," in *Proc. IEEE Custom Integr. Circuits Conf. (CICC)*, Apr. 2022, pp. 1–2.

[11] J. Wang et al., "A 28-nm compute SRAM with bit-serial logic/arithmetic operations for programmable in-memory vector computing," *IEEE J. Solid-State Circuits*, vol. 55, no. 1, pp. 76–86, Jan. 2020.

[12] H. Kim, T. Yoo, T. T. Kim, and B. Kim, "Colonnade: A reconfigurable SRAM-based digital bit-serial compute-in-memory macro for processing neural networks," *IEEE J. Solid-State Circuits*, vol. 56, no. 7, pp. 2221–2233, Jul. 2021.

[13] Y.-D. Chih et al., "An 89 TOPS/W and 16.3 TOPS/mm² all-digital SRAM-based full-precision compute-in memory macro in 22 nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.

[14] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm² fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *Proc. IEEE Int. Solid-State Circuits Conf. (ISSCC)*, vol. 65, Feb. 2022, pp. 1–3.

[15] C.-F. Lee et al., "A 12nm 121-TOPS/W 41.6-TOPS/mm² all digital full precision SRAM-based compute-in-memory with configurable bit-width for AI edge applications," in *Proc. IEEE Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, Jun. 2022, pp. 24–25.

[16] F. Tu et al., "A 28 nm 29.2 TFLOPS/W BF16 and 36.5 TOPS/W INT8 reconfigurable digital CIM processor with unified FP/INT pipeline and bitwise in-memory booth multiplication for cloud deep learning acceleration," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[17] B. Yan et al., "A 1.041-Mb/mm² 27.38-TOPS/W signed-INT8 dynamic-logic-based ADC-less SRAM compute-in-memory macro in 28nm with reconfigurable bitwise operation for AI and embedded applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 188–190.

[18] Y. He et al., "A 28 nm 38-to-102-TOPS/W 8b multiply-less approximate digital SRAM compute-in-memory macro for neural-network inference," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 130–132.

[19] Z. Yue et al., "7.7 CV-CIM: A 28 nm XOR-derived similarity-aware computation-in-memory for cost-volume construction," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 138–139.

[20] J. Yue et al., "A 5.6–89.9TOPS/W heterogeneous computing-in-memory SoC with high-utilization producer-consumer architecture and high-frequency read-free CIM macro," in *Proc. Symp. VLSI Technol. Circuits (VLSI Technol. Circuits)*, 2023, pp. 1–2.

- [21] J. Yue et al., "A 28nm 16.9–300TOPS/W computing-in-memory processor supporting floating-point NN inference/training with intensive-CIM sparse-digital architecture," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Jun. 2023, pp. 1–3.
- [22] G. Desoli et al., "16.7 a 40–310TOPS/W SRAM-based all-digital up to 4b in-memory computing multi-tiled NN accelerator in FD-SOI 18nm for deep-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 260–262.
- [23] M.-E. Shih et al., "20.1 NVE: A 3nm 23.2TOPS/W 12b-digital-CIM-based neural engine for high-resolution visual-quality enhancement on smart devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2024, pp. 360–362.
- [24] K. Goetschalckx and M. Verhelst, "DepFiN: A 12 nm, 3.8TOPs depth-first CNN processor for high res. image processing," in *Proc. Symp. VLSI Circuits*, 2021, pp. 1–2.
- [25] J. S. Park et al., "A multi-mode 8K-MAC HW-utilization-aware neural processing unit with a unified multi-precision datapath in 4 nm flagship mobile SoC," in *IEEE Int. Solid-State Circuits Conf. Dig. Tech. Papers*, Feb. 2022, pp. 246–248.
- [26] C.-Y. Du et al., "A 28 nm 11.2TOPS/W hardware-utilization-aware neural-network accelerator with dynamic dataflow," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, San Francisco, CA, USA, Feb. 2023, pp. 1–3.
- [27] R. Guo et al., "20.2 a 28nm 74.34TFLOPS/W BF16 heterogenous CIM-based accelerator exploiting denoising-similarity for diffusion models," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 362–364.
- [28] P. A. Hager et al., "11.3 metis AIPU: A 12 nm 15TOPS/W 209.6TOPS SoC for Cost- and energy-efficient inference at the edge," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2024, pp. 212–214.
- [29] H. Mori et al., "A 4 nm 6163-TOPS/W/b 4790-TOPS/mm²/b SRAM based digital-computing-in-memory macro supporting bit-width flexibility and simultaneous MAC and weight update," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2023, pp. 132–134.
- [30] C. Banbury et al., "MLPerf tiny benchmark," 2021, *arXiv:2106.07597*.
- [31] H. E. Sumbul, J.-S. Seo, D. H. Morris, and E. Beigne, "A fully digital and row-pipelined compute-in-memory neural network accelerator with system-on-chip-level benchmarking for augmented/virtual reality applications," *IEEE Micro*, vol. 44, no. 2, pp. 61–70, Mar. 2024.
- [32] W. Jiang, C. Caron, P. Avasare, M. Pauwels, M. Verhelst, and W. Dehaene, "HUNBN, a 1.77MB digital in-memory-compute SoC for edge applications achieving 126 TOPs/W (4b) at macro level and 24 TOPs/W at SoC level," in *Proc. IEEE Eur. Solid-State Electron. Res. Conf. (ESSERC)*, Bruges, Belgium, Sep. 2024, pp. 137–140.



Weijie Jiang (Graduate Student Member, IEEE) was born in Hubei, China, in 1996. He received the bachelor's degree in microelectronics from Shanghai Jiao Tong University, Shanghai, China, in 2018, and the master's degree in electronic engineering from KU Leuven (Katholieke Universiteit Leuven), Leuven, Belgium, in 2020, where he is currently pursuing the Ph.D. degree in static random access memory (SRAM) and in-memory-compute circuit design under the supervision of Prof. Wim Dehaene.



Cédric Caron received the bachelor's degree from French Engineering School, ESCPE Lyon, Lyon, France, in 2007.

He is a Chief Engineer with Sony Europe Ltd., Weybridge, U.K. He began his career in December 2010 at Optrima, Brussels, Belgium, a Belgian start-up specializing in 3-D imaging, before joining SoftKinetic, Brussels, in 2011. In 2016, Sony Semiconductors Solutions acquired SoftKinetic to transition it to Sony DepthSensing Solutions. Since 2017, he has been contributing to advancements in hardware–software co-optimization for neural network accelerators at Sony. His professional journey has consistently revolved around the intersection of hardware and software. He started as an embedded software engineer, developing the software required to operate time-of-flight cameras. He joined the Strategic Research Department, Sony DepthSensing Solutions, in 2017, with a focus on leveraging co-optimization techniques to enhance the performance and efficiency of neural network accelerators.



Prabhat Avasare (Member, IEEE) received the Bachelor of Technology degree in electrical engineering from Indian Institute of Technology (Bombay), Mumbai, India, in 1991.

From 1991 to 1996, he was with Tata Research Center, India, and IMEC, Leuven, Belgium, from 1996 to 2013. In 2013, he joined SoftKinetic, Brussels, Belgium, a Belgian start-up specializing in 3-D imaging solutions. In 2016, Sony Semiconductors Solutions acquired SoftKinetic to transition it to Sony DepthSensing Solutions, where he is currently a member of the Technical Staff. He has more than 33 years of experience in Industrial Research and Development—in wide-ranging areas in computer science and electrical engineering such as compiler optimizations, operating systems, hardware–software co-design, reconfigurable hardware, processor architectures, multiobjective optimizations, time-of-flight sensors, and low-power embedded platforms.



Marc Pauwels received the M.Eng. degree in microelectronics from KU Leuven, Leuven, Belgium, in 1984.

He joined IMEC, Leuven, to do research in VLSI system-on-chip design methodologies. In 2010, he joined Optrima, Brussels, Belgium (later SoftKinetic), as a VP of Engineering to develop it from a small start-up into a Research and Development Organization designing full-stack solutions (with depth-sensing ICs, camera hardware/software processing, up to computer vision software and user-experience). He supported the integration process of SoftKinetic into Sony and has been part of the director team at Sony DepthSensing Solutions (SDS), with a focus on research and product development. He is currently the Research and Development Director with SDS. His research interests include hardware/software co-design and bringing innovation into product development.



Marian Verhelst (Fellow, IEEE) received the Ph.D. degree from KU Leuven, Leuven, Belgium, in 2008.

She worked as a Research Scientist at Intel Labs, from 2008 to 2010. She is a Professor with the MICAS Laboratory, KU Leuven, and the Research Director with IMEC, Leuven. Her research focuses on embedded machine learning, hardware accelerators, and low-power edge processing.

Dr. Verhelst received the Laureate Prize of the Royal Academy of Belgium in 2016, the 2021 Intel Outstanding Researcher Award, and the André Mischke YAE Prize for Science and Policy in 2021. She is a scientific advisor to multiple startups, a member of the board of ECSA, and served on the Board of Directors of tinyML. She is a Science Communication Enthusiast as an IEEE SSCS Distinguished Lecturer, a Regular Member of the Nerdland Science Podcast (Dutch), and the Founding Mother of KU Leuven's Innovation Lab High School Program.



Wim Dehaene (Senior Member, IEEE) received the master's and Ph.D. degrees from KU Leuven, Leuven, Belgium, in 1991 and 1996, respectively.

In-memory-compute is an integral part of this. Part of his research is performed in cooperation with IMEC, Leuven, where he is also a part-time Principal Scientist. He is also leading the circuit design aspects in several biomedical IC design projects. He is currently a Full Professor and the Head of the MICAS Division, KU Leuven. His research domain is the circuit-level design of digital circuits. His research interests include ultralow-power signal processing and memories in advanced CMOS technologies.

Dr. Dehaene is a member of the ESSERC Steering Committee. He was the General Chair of ESSERC 2024. He has also served for several years on the ISSCC Program Committee.